

UNIVERSITÀ POLITECNICA DELLE MARCHE
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

ShieldUp



Corso di
PROGRAMMAZIONE MOBILE
Anno accademico 2025-2026

Studenti:

Lorenzo Califano

Chiara Carlomagno

Youssef Lafsahi

Professore:

Alessandro Storti



Dipartimento di Ingegneria dell'Informazione



Indice

Introduzione	2
0.1 Introduzione	3
0.2 Analisi dei requisiti	3
0.2.1 Requisiti funzionali	3
0.2.2 Requisiti non funzionali	4
0.2.3 Casi d'uso	4
0.3 Kotlin	11
0.3.1 Architettura	11
0.3.2 Database	12
0.3.3 UI	16
0.3.4 Sviluppo	22
0.3.5 Testing	35
0.4 Flutter	38
0.4.1 Architettura	38
0.4.2 Database	40
0.4.3 UI	40
0.4.4 Sviluppo	47
0.4.5 Testing	55
0.5 Confronto tra le implementazioni	56
0.6 Conclusioni	57

0.1 Introduzione

L'obiettivo del progetto è stato lo sviluppo di **ShieldUp**, un'applicazione mobile dedicata alla sicurezza personale. L'applicazione mette a disposizione una serie di strumenti utilizzabili in situazioni di emergenza o di potenziale pericolo, tra cui la chiamata rapida ai servizi di soccorso, la segnalazione di aree pericolose, la gestione dei contatti di emergenza e la simulazione di una chiamata telefonica.

Per il progetto sono state realizzate due implementazioni dell'applicazione:

- una versione **nativa Android**, sviluppata utilizzando il linguaggio Kotlin;
- una versione **cross-platform**, sviluppata mediante il framework Flutter e il linguaggio Dart.

Le due applicazioni condividono gli stessi requisiti funzionali e la stessa logica di utilizzo, differenziandosi esclusivamente per le tecnologie impiegate durante lo sviluppo. Nei capitoli successivi vengono descritti i requisiti del progetto e, successivamente, le due implementazioni, illustrandone l'architettura, la progettazione del database, l'interfaccia utente, le principali funzionalità sviluppate e le attività di testing effettuate.

0.2 Analisi dei requisiti

Prima dello sviluppo dell'applicazione è stata effettuata un'analisi dei requisiti, con l'obiettivo di individuare le funzionalità che il sistema avrebbe dovuto offrire agli utenti. L'applicazione è stata progettata per fornire strumenti semplici e immediatamente accessibili, privilegiando un'interfaccia intuitiva e tempi di interazione ridotti.

0.2.1 Requisiti funzionali

L'applicazione deve consentire all'utente di:

- registrare un nuovo account ed effettuare il login;
- gestire il proprio profilo personale;
- aggiungere, modificare ed eliminare contatti di emergenza;
- effettuare rapidamente una chiamata ai servizi di emergenza (112);
- visualizzare una mappa interattiva;
- segnalare situazioni di pericolo indicando posizione, titolo e descrizione;
- visualizzare le segnalazioni presenti sulla mappa;
- utilizzare la posizione GPS corrente durante l'inserimento di una segnalazione;
- simulare una chiamata telefonica programmabile;
- distinguere utenti standard e psicologi mediante differenti tipologie di account.

0.2.2 Requisiti non funzionali

Oltre alle funzionalità offerte, sono stati considerati alcuni requisiti non funzionali fondamentali.

- L'interfaccia deve risultare semplice e facilmente utilizzabile anche in situazioni di stress.
- Le operazioni principali devono essere raggiungibili con un numero ridotto di interazioni.
- I dati degli utenti devono essere memorizzati in maniera persistente.
- L'applicazione deve utilizzare i servizi di localizzazione solamente quando necessario.
- Il sistema deve garantire una navigazione coerente tra le diverse schermate.
- Le due implementazioni devono offrire funzionalità equivalenti pur utilizzando tecnologie differenti.

0.2.3 Casi d'uso

I casi d'uso riportati fanno riferimento alle funzionalità presenti nella versione finale consegnata dell'applicazione. Alcune funzionalità progettate inizialmente sono state semplificate o non incluse nella versione Flutter; tali aspetti verranno discussi nei capitoli dedicati allo sviluppo e alle problematiche riscontrate.

UC1 - Registrazione utente

Campo	Descrizione
Attore	Utente / Psicologo
Descrizione	L'utente crea un nuovo account compilando il modulo di registrazione.
Precondizioni	L'utente non possiede già un account associato all'indirizzo e-mail inserito.
Scenario principale	1. L'utente apre la schermata di registrazione. 2. Inserisce nome, cognome, e-mail e password. 3. Seleziona il tipo di account. 4. Conferma la registrazione. 5. Il sistema salva i dati nel database.
Postcondizioni	L'account viene creato ed è possibile effettuare il login.

Table 1: *Caso d'uso UC1 - Registrazione utente*

UC2 - Login utente

Campo	Descrizione
Attore	Utente
Descrizione	L'utente accede all'applicazione utilizzando le proprie credenziali.
Precondizioni	L'utente possiede un account registrato.
Scenario principale	1. L'utente apre la schermata di login. 2. Inserisce e-mail e password. 3. Il sistema verifica le credenziali. 4. Viene aperta la schermata principale dell'applicazione.
Postcondizioni	L'utente è autenticato e può utilizzare tutte le funzionalità disponibili.

Table 2: *Caso d'uso UC2 - Login utente*

UC3 - Login psicologo

Campo	Descrizione
Attore	Psicologo
Descrizione	Lo psicologo accede all'area dedicata mediante le proprie credenziali.
Precondizioni	Lo psicologo possiede un account registrato nel sistema.
Scenario principale	1. Lo psicologo apre la schermata di login. 2. Inserisce e-mail e password. 3. Il sistema verifica le credenziali. 4. Viene aperta la dashboard riservata agli psicologi.
Postcondizioni	Lo psicologo può visualizzare e gestire le richieste di supporto ricevute e aprire una nuova finestra di prenotazioni.

Table 3: *Caso d'uso UC3 - Login psicologo*

UC4 - Gestione profilo

Campo	Descrizione
Attore	Utente
Descrizione	L'utente visualizza e modifica le informazioni del proprio profilo personale.
Precondizioni	L'utente ha effettuato il login.
Scenario principale	1. L'utente accede alla sezione Profilo. 2. Visualizza le informazioni personali. 3. Modifica i dati desiderati. 4. Conferma le modifiche. 5. Il sistema aggiorna il database.
Postcondizioni	I dati del profilo risultano aggiornati.

Table 4: *Caso d'uso UC4 - Gestione profilo*

UC5 - Gestione contatti di emergenza

Campo	Descrizione
Attore	Utente
Descrizione	L'utente gestisce l'elenco dei propri contatti di emergenza.
Precondizioni	L'utente ha effettuato il login.
Scenario principale	1. L'utente apre la sezione Contatti di emergenza. 2. Inserisce nome e numero di telefono del contatto. 3. Il sistema verifica la validità dei dati. 4. Il contatto viene salvato nel database. 5. L'utente può eliminare uno o più contatti.
Postcondizioni	L'elenco dei contatti di emergenza risulta aggiornato.

Table 5: *Caso d'uso UC5 - Gestione contatti di emergenza*

UC6 - Chiamata ai servizi di emergenza

Campo	Descrizione
Attore	Utente
Descrizione	L'utente avvia rapidamente una chiamata verso il numero di emergenza 112.
Precondizioni	Il dispositivo dispone della funzionalità telefonica.
Scenario principale	1. L'utente seleziona la funzione "Chiama soccorsi". 2. Viene avviato un conto alla rovescia. 3. L'utente può annullare l'operazione. 4. Al termine del conto alla rovescia viene aperto il dialer con il numero 112.
Postcondizioni	Il dispositivo è pronto ad effettuare la chiamata di emergenza.

Table 6: *Caso d'uso UC6 - Chiamata ai servizi di emergenza*

UC7 - Segnalazione di un pericolo

Campo	Descrizione
Attore	Utente
Descrizione	L'utente inserisce una nuova segnalazione geolocalizzata.
Precondizioni	L'utente ha effettuato il login e ha concesso l'accesso alla posizione.
Scenario principale	1. L'utente apre la mappa. 2. Seleziona "Segnala pericolo". 3. Inserisce titolo e descrizione. 4. Utilizza la posizione corrente oppure seleziona manualmente un punto sulla mappa. 5. Il sistema salva la segnalazione.
Postcondizioni	La segnalazione è disponibile sulla mappa per gli altri utenti.

Table 7: *Caso d'uso UC7 - Segnalazione di un pericolo*

UC8 - Visualizzazione delle segnalazioni sulla mappa

Campo	Descrizione
Attore	Utente
Descrizione	L'utente consulta la mappa per visualizzare le segnalazioni inserite dagli altri utenti.
Precondizioni	L'utente ha effettuato il login.
Scenario principale	1. L'utente apre la sezione Mappa. 2. Il sistema carica le segnalazioni presenti nel database. 3. Le segnalazioni vengono visualizzate mediante marker sulla mappa.
Postcondizioni	L'utente ha consultato le segnalazioni presenti nell'area di interesse.

Table 8: *Caso d'uso UC8 - Visualizzazione delle segnalazioni*

UC9 - Chiamata simulata

Campo	Descrizione
Attore	Utente
Descrizione	L'utente programma una chiamata simulata per uscire da una situazione di disagio.
Precondizioni	L'utente ha effettuato il login.
Scenario principale	1. L'utente seleziona la funzione "Chiamata simulata". 2. Inserisce il nome del chiamante e il tempo di attesa. 3. Conferma la programmazione. 4. Allo scadere del tempo viene simulata una chiamata in arrivo.
Postcondizioni	L'utente riceve la chiamata simulata secondo le impostazioni selezionate.

Table 9: *Caso d'uso UC9 - Chiamata simulata*

UC10 - Richiesta di supporto psicologico

Campo	Descrizione
Attore	Utente
Descrizione	L'utente invia una richiesta di supporto psicologico attraverso l'applicazione.
Precondizioni	L'utente ha effettuato il login.
Scenario principale	1. L'utente accede alla sezione Supporto. 2. Invia la richiesta di supporto. 3. Il sistema salva la richiesta nel database.
Postcondizioni	La richiesta è disponibile per gli psicologi registrati.

Table 10: *Caso d'uso UC10 - Richiesta di supporto psicologico*

UC11 - Gestione delle richieste di supporto

Campo	Descrizione
Attore	Psicologo
Descrizione	Lo psicologo consulta e gestisce le richieste di supporto ricevute dagli utenti.
Precondizioni	Lo psicologo ha effettuato il login.
Scenario principale	1. Lo psicologo accede alla dashboard. 2. Visualizza l'elenco delle richieste. 3. Seleziona una richiesta. 4. Aggiorna lo stato della richiesta.
Postcondizioni	La richiesta risulta aggiornata nel sistema.

Table 11: *Caso d'uso UC11 - Gestione delle richieste di supporto*

UC12 - Logout

Campo	Descrizione
Attore	Utente / Psicologo
Descrizione	L'utente termina la sessione corrente.
Precondizioni	È presente una sessione autenticata.
Scenario principale	1. L'utente seleziona la funzione Logout. 2. Il sistema termina la sessione corrente. 3. Viene visualizzata la schermata di login.
Postcondizioni	L'utente risulta disconnesso dall'applicazione.

Table 12: *Caso d'uso UC12 - Logout*

0.3 Kotlin

La versione Android di ShieldUp è stata sviluppata in Kotlin utilizzando Android Studio. L'applicazione è basata su una struttura a singola Activity principale, **MainActivity**, che ospita le diverse schermate tramite il componente di navigazione di Jetpack. Le funzionalità dell'app sono organizzate in package separati, ciascuno dedicato ad una specifica area applicativa.

0.3.1 Architettura

L'architettura dell'applicazione Android è organizzata intorno a **MainActivity**, che svolge il ruolo di contenitore principale dell'interfaccia. All'interno di essa viene gestita la navigazione tra i vari fragment e viene mostrata una barra inferiore differente in base al ruolo dell'utente autenticato. In particolare, gli utenti standard visualizzano una barra con accesso a Home, Mappa, Supporto, Chat e Profilo, mentre gli psicologi visualizzano una barra dedicata con Dashboard, Chat e Profilo.

Il progetto è suddiviso nei seguenti package principali:

- **data**: contiene **FirestoreRepository**, classe centrale per l'accesso a Cloud Firestore, e i DTO utilizzati per rappresentare utenti, contatti di emergenza, red zone, disponibilità, richieste urgenti, chat e messaggi.
- **utils**: contiene **SessionManager**, utilizzato per salvare localmente i dati della sessione tramite **SharedPreferences**.
- **ui.auth**: contiene i fragment dedicati a login e registrazione.
- **ui.home**: contiene la schermata principale dell'utente standard, dalla quale si accede rapidamente alle funzioni principali.
- **ui.contacts**: gestisce l'inserimento, la visualizzazione e l'eliminazione dei contatti di emergenza.
- **ui.emergency**: contiene la schermata di conto alla rovescia per l'attivazione dell'SOS.
- **ui.redzones**: gestisce la mappa, le segnalazioni di pericolo, la lista delle red zone e il calcolo del percorso sicuro.
- **ui.followme**: contiene la logica per la funzionalità Seguimi a casa, basata sull'aggiornamento periodico della posizione GPS.
- **ui.fakecall**: contiene la schermata dedicata alla chiamata simulata.
- **ui.psychologist**: contiene la dashboard dello psicologo, utilizzata per la gestione delle disponibilità e delle richieste urgenti.
- **ui.chat**: contiene la lista delle chat e la schermata di conversazione.
- **ui.settings**: gestisce profilo, modifica dei dati personali e logout.

L'applicazione non utilizza adapter o `RecyclerView`; le liste dinamiche vengono costruite inserendo programmaticamente viste all'interno di `LinearLayout` verticali. Questa scelta ha semplificato lo sviluppo delle schermate, anche se risulta meno efficiente rispetto all'uso di componenti specifici per liste di grandi dimensioni.

La persistenza remota dei dati è affidata a Firebase Cloud Firestore, mentre la sessione dell'utente viene mantenuta localmente tramite `SharedPreferences`. Per le funzionalità geografiche vengono utilizzati Google Maps SDK, Fused Location Provider e Google Directions API.

0.3.2 Database

Per la memorizzazione dei dati l'applicazione utilizza Firebase Cloud Firestore. Il database è organizzato in collection principali e sotto-collezioni, in modo da separare le diverse entità gestite dall'applicazione.

La classe centrale per l'accesso ai dati è `FirestoreRepository`, che contiene i metodi per leggere, scrivere, aggiornare ed eliminare i documenti presenti su Firestore. Per la funzionalità Seguimi a casa è inoltre presente `FollowMeRepository`, dedicata alla gestione delle sessioni di localizzazione live.

Le principali collection utilizzate sono:

- **users**: contiene gli account degli utenti e degli psicologi;
- **users/{userId}/emergencyContacts**: contiene i contatti di emergenza associati ad ogni utente;
- **redZones**: contiene le segnalazioni di pericolo visualizzate sulla mappa;
- **availabilities**: contiene le disponibilità inserite dagli psicologi;
- **urgentPsychologicalRequests**: contiene le richieste urgenti di supporto psicologico;
- **chats**: contiene le conversazioni tra utenti e psicologi;
- **chats/{chatId}/messages**: contiene i messaggi di ogni chat;
- **liveLocations**: contiene le sessioni di condivisione della posizione live.

Collection users

La collection **users** memorizza le informazioni anagrafiche e di accesso degli utenti. Ogni documento rappresenta un account e contiene i seguenti campi:

- **name**: nome dell'utente;
- **surname**: cognome dell'utente;
- **email**: indirizzo e-mail;
- **password**: password dell'utente;
- **role**: ruolo dell'account, cioè **STANDARD** oppure **PSYCHOLOGIST**;

- **createdAt:** timestamp di creazione dell'account.

Questa collection viene utilizzata durante la registrazione, il login, la modifica dei dati personali e il caricamento degli psicologi disponibili. L'autenticazione è stata implementata manualmente tramite Firestore, senza utilizzare Firebase Authentication.

Sotto-collezione **emergencyContacts**

Per ogni utente standard è presente una sotto-collezione:

users/{userId}/emergencyContacts

Ogni documento rappresenta un contatto di emergenza e contiene:

- **name:** nome del contatto;
- **phone:** numero di telefono;
- **createdAt:** timestamp di creazione.

Questi dati vengono utilizzati dalla funzione SOS e dalla funzione Seguimi a casa per inviare messaggi ai contatti fidati dell'utente.

Collection **redZones**

La collection **redZones** contiene le segnalazioni di pericolo inserite dagli utenti. Ogni documento contiene:

- **title:** titolo della segnalazione;
- **description:** descrizione del pericolo;
- **latitude:** latitudine;
- **longitude:** longitudine;
- **createdAt:** timestamp di creazione;
- **userId:** identificativo dell'utente che ha creato la segnalazione;
- **type:** tipologia del pericolo.

Le segnalazioni vengono mostrate sulla mappa come marker e come aree circolari rosse. L'applicazione considera attive solo le segnalazioni create nell'ultima ora.

Collection `availabilities`

La collection `availabilities` viene utilizzata dagli psicologi per pubblicare le proprie disponibilità. I campi principali sono:

- `psychologistId`: identificativo dello psicologo;
- `psychologistName`: nome dello psicologo;
- `scheduledAt`: timestamp dello slot;
- `date`: data in formato testuale;
- `time`: ora in formato testuale;
- `type`: tipo di consulto;
- `booked`: indica se lo slot è stato prenotato;
- `createdAt`: timestamp di creazione.

Quando un utente prenota uno slot, vengono aggiunti anche i dati relativi all'utente prenotato.

Collection `urgentPsychologicalRequests`

La collection `urgentPsychologicalRequests` contiene le richieste urgenti di supporto psicologico. Ogni richiesta include:

- `userId`: identificativo dell'utente;
- `userName`: nome dell'utente;
- `status`: stato della richiesta, ad esempio `WAITING` o `ACCEPTED`;
- `createdAt`: timestamp di creazione;
- `psychologistId`: identificativo dello psicologo che accetta la richiesta;
- `psychologistName`: nome dello psicologo;
- `chatId`: identificativo della chat creata;
- `acceptedAt`: timestamp di accettazione.

Quando uno psicologo accetta una richiesta urgente, il sistema aggiorna lo stato della richiesta e crea automaticamente una chat associata.

Collection chats e sotto-collezione messages

La collection `chats` rappresenta le conversazioni tra utenti e psicologi. Ogni documento contiene:

- `userId`: identificativo dell'utente;
- `userName`: nome dell'utente;
- `psychologistId`: identificativo dello psicologo;
- `psychologistName`: nome dello psicologo;
- `lastMessage`: ultimo messaggio inviato;
- `lastMessageAt`: timestamp dell'ultimo messaggio;
- `createdAt`: timestamp di creazione della chat.

I messaggi sono salvati nella sotto-collezione:

`chats/{chatId}/messages`

Ogni messaggio contiene mittente, testo e timestamp di invio.

Collection liveLocations

La collection `liveLocations` viene utilizzata per la funzionalità Seguiami a casa. Ogni documento rappresenta una sessione di condivisione della posizione e contiene:

- `userId`: identificativo dell'utente;
- `userName`: nome dell'utente;
- `destination`: destinazione inserita;
- `latitude`: latitudine corrente;
- `longitude`: longitudine corrente;
- `active`: indica se la sessione è attiva;
- `createdAt` o `startedAt`: timestamp di avvio;
- `updatedAt`: timestamp dell'ultimo aggiornamento;
- `expiresAt`: timestamp di scadenza.

Durante una sessione attiva, la posizione viene aggiornata periodicamente ogni cinque secondi e può essere visualizzata tramite una web app collegata allo stesso database Firebase.

0.3.3 UI

Nella progettazione dell'interfaccia grafica è stato scelto di adottare uno stile semplice ed intuitivo, privilegiando la rapidità di accesso alle funzionalità di emergenza. L'intera applicazione è stata sviluppata seguendo le linee guida di Material Design, utilizzando componenti standard del framework Android come `Button`, `EditText`, `TextView`, `ScrollView` e `LinearLayout`.

La navigazione dell'applicazione è organizzata mediante una singola `MainActivity` che ospita tutti i fragment. In base al ruolo dell'utente autenticato viene visualizzata una barra di navigazione inferiore differente, consentendo di mostrare solamente le funzionalità disponibili per quello specifico profilo.

L'interfaccia è stata progettata cercando di ridurre il numero di operazioni necessarie per raggiungere le funzioni principali, in particolare quelle dedicate alle situazioni di emergenza.

Tutti gli screenshot riportati nella relazione sono stati acquisiti utilizzando il dispositivo principale usato durante le attività di sviluppo e testing, un Google Pixel con Android 10.

Home

La schermata Home rappresenta il punto di accesso principale per gli utenti standard. Al suo interno sono presenti cinque pulsanti di grandi dimensioni che permettono di accedere rapidamente alle funzionalità principali dell'applicazione:

- chiamata ai soccorsi;
- segnalazione di un pericolo;
- funzione Seguimi a casa;
- calcolo del percorso sicuro;
- chiamata simulata.

La disposizione delle funzioni è stata studiata per privilegiare la rapidità di utilizzo, evitando menu intricati o procedure particolarmente lunghe. Tutte le operazioni più importanti risultano infatti raggiungibili con un singolo tocco.



Mappa

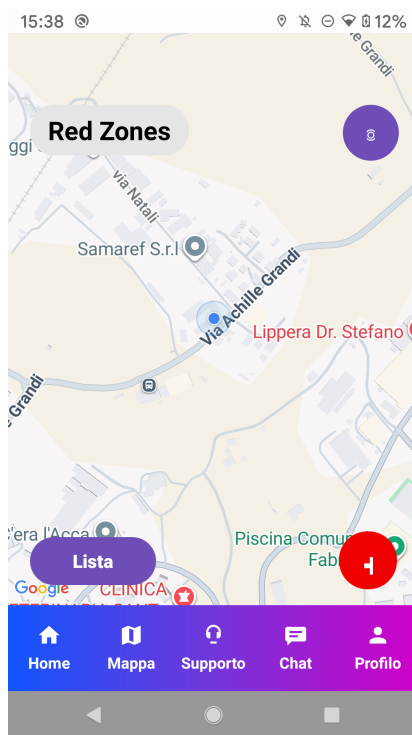
La schermata della mappa rappresenta uno degli elementi centrali dell'applicazione. Essa integra Google Maps SDK e consente sia la consultazione delle segnalazioni esistenti sia l'inserimento di nuove Red Zone.

Ogni segnalazione viene rappresentata mediante un marker e da un'area circolare evidenziata in rosso, che permette all'utente di identificare immediatamente la posizione del pericolo.

Dalla stessa schermata è inoltre possibile:

- visualizzare la propria posizione corrente;
- inserire una nuova segnalazione;
- consultare l'elenco delle segnalazioni recenti;

L'interfaccia è stata progettata cercando di concentrare tutte le funzionalità geografiche in un'unica schermata, evitando continui cambi di pagina durante l'utilizzo.



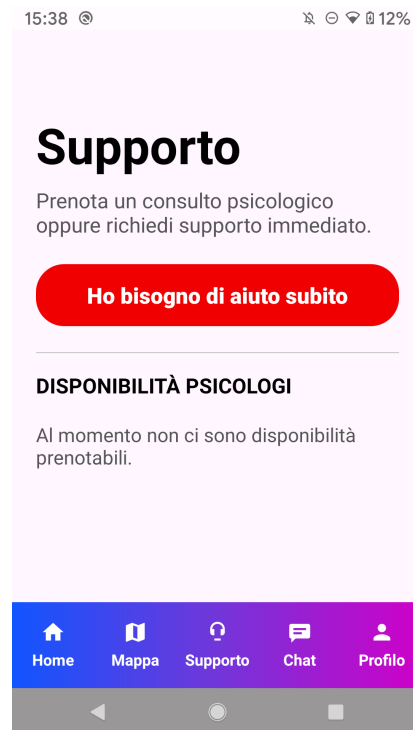
Supporto psicologico

La schermata dedicata al supporto psicologico permette agli utenti di richiedere assistenza secondo due modalità differenti.

La prima consiste nell'invio di una richiesta urgente, destinata agli psicologi disponibili in quel momento.

La seconda modalità consente invece di prenotare uno degli slot orari messi a disposizione dagli psicologi, scegliendo tra le disponibilità future presenti nel database.

L'interfaccia mostra solamente gli slot ancora disponibili e permette di effettuare la prenotazione con un singolo pulsante.

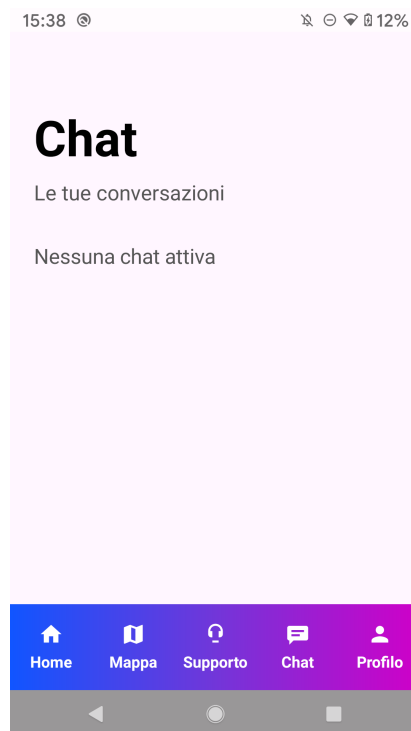


Chat

La funzionalità di messaggistica privata è composta da due schermate distinte.

La prima mostra l'elenco delle conversazioni attive dell'utente, mentre la seconda permette lo scambio dei messaggi con lo psicologo associato alla chat.

Ogni conversazione visualizza il nome dell'interlocutore e l'ultimo messaggio inviato, consentendo di individuare rapidamente la chat desiderata.



Dashboard dello psicologo

Gli utenti registrati come psicologi visualizzano una schermata iniziale differente rispetto agli utenti standard.

La dashboard consente di gestire le proprie disponibilità, inserendo nuovi slot orari destinati ai consulti futuri, e di visualizzare in tempo reale le richieste urgenti provenienti dai pazienti.

Da questa schermata è inoltre possibile accettare una richiesta di supporto immediato, operazione che genera automaticamente una nuova conversazione privata tra psicologo e utente.



0.3.4 Sviluppo

Lo sviluppo dell'applicazione è stato suddiviso in moduli indipendenti, ciascuno dedicato ad una specifica funzionalità. Questa scelta ha permesso di sviluppare e testare ogni componente separatamente, semplificando la manutenzione del progetto.

Nei paragrafi seguenti vengono descritte le principali componenti implementate.

Sistema di autenticazione

Il sistema di autenticazione è stato implementato senza ricorrere a Firebase Authentication. Le credenziali degli utenti vengono infatti memorizzate direttamente all'interno della collection `users` di Cloud Firestore.

Durante la registrazione vengono effettuati diversi controlli di validazione sui dati inseriti. In particolare viene verificato che tutti i campi siano compilati, che l'indirizzo e-mail non sia già presente nel database e che la password soddisfi alcuni requisiti minimi di sicurezza, tra cui una lunghezza di almeno otto caratteri e la presenza di almeno una lettera maiuscola e di un numero.

Una volta completata la registrazione, il nuovo documento viene salvato nel database e le informazioni principali dell'utente vengono memorizzate localmente mediante `SharedPreferences`, evitando così di richiedere un nuovo login ad ogni apertura dell'applicazione.

Durante la fase di accesso viene invece effettuata una ricerca dell'utente tramite indirizzo e-mail. Se la password inserita coincide con quella presente nel database, la sessione viene ripristinata e l'utente viene automaticamente indirizzato verso l'interfaccia corrispondente al proprio ruolo.

Gestione dei contatti di emergenza

Ogni utente può memorizzare una lista di contatti fidati da utilizzare durante le situazioni di emergenza.

L'inserimento di un nuovo contatto avviene attraverso un'apposita schermata che richiede nome e numero di telefono. I dati vengono salvati sia all'interno della sotto-collezione dedicata su Firestore sia localmente mediante `SharedPreferences`, consentendo di recuperarli rapidamente anche durante le operazioni di emergenza.

L'utente può inoltre eliminare in qualsiasi momento uno o più contatti precedentemente inseriti. La lista mostrata nell'interfaccia viene aggiornata automaticamente dopo ogni operazione di inserimento o cancellazione.

Chiamata di emergenza

Quando l'utente seleziona il pulsante dedicato viene visualizzato un conto alla rovescia della durata di dieci secondi. Questo intervallo consente di annullare rapidamente l'operazione nel caso in cui il pulsante sia stato premuto accidentalmente.

Se il timer raggiunge lo zero vengono eseguite diverse operazioni in sequenza.

In primo luogo viene acquisita la posizione geografica del dispositivo. Successivamente viene generato un link di Google Maps contenente le coordinate rilevate e tale collegamento viene inviato tramite SMS a tutti i contatti di emergenza precedentemente registrati.

Infine viene aperto automaticamente il dialer del dispositivo con il numero 112 già compilato, permettendo all'utente di avviare immediatamente la chiamata ai servizi di emergenza.

Nel caso in cui i permessi relativi alla posizione o agli SMS non siano disponibili, l'applicazione continua comunque ad aprire il dialer garantendo il funzionamento della parte più importante della procedura.

Gestione delle Red Zone

Le Red Zone rappresentano le segnalazioni di pericolo condivise dagli utenti.

Ogni segnalazione è composta da un titolo, una descrizione e da una posizione geografica che può corrispondere sia alla posizione corrente dell'utente sia ad un punto selezionato manualmente sulla mappa.

Una volta salvata, la segnalazione viene immediatamente memorizzata su Firestore e resa disponibile a tutti gli altri utenti dell'applicazione.

Per evitare la permanenza di informazioni non più attuali, l'applicazione visualizza esclusivamente le segnalazioni create nell'ultima ora.

Calcolo del percorso sicuro

Una delle funzionalità più complesse sviluppate durante il progetto è il calcolo del percorso sicuro. L'obiettivo non è semplicemente trovare il tragitto più breve tra due punti, ma individuare un percorso che riduca il più possibile il passaggio attraverso le redzones.

La funzionalità è stata implementata all'interno del package `ui.redzones`, principalmente attraverso le classi `RedZonesFragment`, `SafeRouteService`, `RouteSafetyUtils` e `PolylineDecoder`.

Quando l'utente seleziona la funzione Percorso sicuro, viene richiesto l'indirizzo di destinazione tramite un'apposita dialog window. Successivamente l'applicazione recupera la posizione corrente del dispositivo utilizzando il servizio `FusedLocationProviderClient`. Una volta ottenute origine e destinazione viene interrogata la Google Directions API per ottenere uno o più percorsi pedonali.

```
1 safeRouteService.calculateWalkingRoutes(  
2     apiKey = BuildConfig.MAPS_API_KEY,  
3     origin = origin,  
4     destinationAddress = destinationAddress,  
5     onSuccess = { routes ->  
6         chooseAndDrawSafestRoute(routes, origin, destinationAddress)  
7     }  
8 )
```

Listing 1: Avvio del calcolo del percorso

La Google Directions API restituisce una o più possibili alternative di percorso. Ogni percorso viene convertito in una sequenza di coordinate geografiche (**LatLng**) mediante la classe **PolylineDecoder**, che decodifica la polilinea compressa restituita dal servizio Google.

Successivamente ogni percorso viene analizzato confrontandolo con tutte le Red Zone attualmente attive. Per ciascun tragitto vengono calcolati due parametri:

- il numero di intersezioni con le aree di pericolo;
- la lunghezza complessiva del percorso.

I percorsi vengono quindi ordinati privilegiando innanzitutto quello con il minor numero di attraversamenti di aree pericolose e, a parità di rischio, quello con distanza minore.

```
1 .sortedWith(  
2     compareBy<SafeRouteResult> {  
3         it.dangerousIntersections  
4     }.thenBy {  
5         it.distanceMeters  
6     }  
7 )
```

Listing 2: Valutazione dei percorsi

In questo modo il criterio di sicurezza ha sempre priorità rispetto alla lunghezza del percorso. Un tragitto leggermente più lungo viene preferito se consente di evitare completamente le aree considerate pericolose.

Nel caso in cui tutti i percorsi iniziali attraversino almeno una Red Zone, il sistema non si limita a scegliere il meno rischioso, ma tenta automaticamente di costruire percorsi alternativi.

Per ciascuna delle prime due aree di pericolo intercettate vengono generati una serie di waypoint disposti radialmente attorno alla zona critica. Tali punti vengono utilizzati come tappe intermedie in nuove richieste alla Google Directions API.

```
1 RouteSafetyUtils  
2     .createAvoidanceWaypointsAroundZone(  
3         zoneCenter = center,  
4         radiusMeters = redZoneRadiusMeters  
5     )
```

Listing 3: Generazione dei waypoint

La funzione genera otto waypoint distribuiti lungo una circonferenza avente raggio pari a circa 2.5 volte quello della Red Zone. In questo modo vengono costruite automaticamente diverse possibili deviazioni attorno all'area considerata pericolosa.

Per ogni insieme di waypoint viene effettuata una nuova interrogazione delle Google Directions API. Tutti i percorsi ottenuti vengono nuovamente analizzati mediante gli stessi criteri di sicurezza, selezionando infine quello con il minor numero di intersezioni e la minore lunghezza.

Il percorso selezionato viene infine disegnato sulla mappa tramite una polilinea verde e viene evidenziata la destinazione con un apposito marker.

```
1 routePolyline = map.addPolyline(  
2     PolylineOptions()  
3     .addAll(points)  
4     .width(10f)  
5     .color(0xFF2E7D32.toInt())  
6 )
```

Listing 4: Disegno del percorso

La funzionalità è stata divisa in più file, ognuno con una responsabilità specifica. **RedZonesFragment** coordina l'interfaccia grafica, la mappa, il caricamento delle Red Zone e l'avvio del calcolo. **SafeRouteService** si occupa invece della comunicazione con Google Directions API, costruendo la richiesta HTTP e restituendo i percorsi decodificati. La classe **PolylineDecoder** converte le polilinee compresse restituite da Google in liste di coordinate **LatLng**. Infine, **RouteSafetyUtils** contiene la parte algoritmica vera e propria, cioè il calcolo delle distanze, il conteggio delle intersezioni con le Red Zone e la generazione dei waypoint di deviazione.

Per stabilire se un percorso attraversa una zona pericolosa, il sistema non controlla solamente se un punto del percorso coincide con una Red Zone, ma calcola la distanza minima tra ogni segmento del tragitto e il centro della zona rossa. Un segmento è costituito da due punti consecutivi della polyline restituita da Google.

```
1 fun countDangerousIntersections(  
2     routePoints: List<LatLng>,  
3     redZones: List<RedZoneDto>,  
4     redZoneRadiusMeters: Double  
5 ): Int {  
6     return redZones.count { zone ->  
7         val zoneCenter = LatLng(zone.latitude, zone.longitude)  
8  
9         routePoints.zipWithNext().any { segment ->  
10             distancePointToSegmentMeters(  
11                 point = zoneCenter,  
12                 start = segment.first,  
13                 end = segment.second  
14             ) <= redZoneRadiusMeters  
15         }  
16     }  
17 }
```

Listing 5: Conteggio delle intersezioni con le Red Zone

La funzione sopra rappresenta il punto centrale dell'algoritmo di valutazione del rischio. L'obiettivo non è infatti verificare solamente se un punto del percorso coincide con il centro di una Red Zone, ma controllare se il percorso attraversa realmente l'area di pericolo.

La funzione scorre tutte le Red Zone presenti sulla mappa e, per ciascuna di esse, suddivide il percorso nei segmenti che collegano due coordinate consecutive della polilinea restituita da Google Directions API. Questo viene realizzato mediante il metodo **zipWithNext()**, che permette di ottenere in modo molto semplice tutte le coppie di punti adiacenti.

Per ogni segmento viene richiamata la funzione **distancePointToSegmentMeters()**, che restituisce la distanza minima tra il centro della Red Zone e il segmento stesso. Se almeno uno dei segmenti passa entro il raggio della zona di pericolo, quella Red Zone viene considerata attraversata e viene incrementato il numero di intersezioni.

Questa soluzione è risultata più precisa rispetto ad un semplice controllo sui singoli punti della polilinea, poiché un segmento potrebbe attraversare una Red Zone anche quando nessuna delle coordinate campionate ricade esattamente al suo interno.

La parte più delicata dell'algoritmo è il calcolo della distanza tra il centro della Red Zone e un segmento del percorso. Per effettuare questa operazione, le coordinate geografiche vengono convertite in coordinate piane locali tramite le funzioni `toX` e `toY`. Successivamente viene calcolata la proiezione ortogonale del punto sul segmento.

```

1 val t = (((pointX - startX) * dx) +
2         ((pointY - startY) * dy)) / (dx * dx + dy * dy)
3
4 val clamped = t.coerceIn(0.0, 1.0)
5
6 val closestX = startX + clamped * dx
7 val closestY = startY + clamped * dy
8
9 return sqrt((pointX - closestX).pow(2.0) +
10            (pointY - closestY).pow(2.0))

```

Listing 6: Distanza tra una Red Zone e un segmento del percorso

Dopo aver convertito latitudine e longitudine in coordinate cartesiane locali, viene calcolata la proiezione ortogonale del centro della Red Zone sul segmento del percorso. Il valore `t` rappresenta la posizione della proiezione lungo il segmento: un valore pari a 0 indica il punto iniziale, un valore pari a 1 il punto finale, mentre valori compresi tra 0 e 1 individuano punti interni al segmento.

L'utilizzo del metodo `coerceIn(0.0, 1.0)` è fondamentale, poiché impedisce che la proiezione venga calcolata sulla retta infinita anziché sul segmento effettivo. In questo modo il punto più vicino viene sempre ricondotto ad una posizione realmente appartenente al percorso.

Infine viene calcolata la distanza tra il centro della Red Zone e il punto appena individuato. Tale valore viene confrontato con il raggio della zona di pericolo (160 metri); se la distanza risulta inferiore al raggio, il segmento viene considerato come attraversamento della Red Zone.

Quando il percorso migliore attraversa una o più Red Zone, il sistema tenta di costruire percorsi alternativi. Per ogni zona intersecata vengono generati otto waypoint disposti radialmente attorno al centro del pericolo, con angoli distanziati di 45 gradi.

```

1 val distance = radiusMeters * 2.6
2 val angles = listOf(0, 45, 90, 135, 180, 225, 270, 315)
3
4 return angles.map { angle ->
5     val radians = Math.toRadians(angle.toDouble())
6
7     LatLng(
8         zoneCenter.latitude +
9             (sin(radians) * distance / metersPerLat),
10        zoneCenter.longitude +
11            (cos(radians) * distance / metersPerLng)
12    )
13 }

```

Listing 7: Waypoint generati intorno alla Red Zone

L'idea nasce dal fatto che Google Directions API non mette a disposizione un parametro che permetta di indicare aree da evitare durante il calcolo del percorso pedonale. Per superare questa limitazione è stata adottata una soluzione alternativa: vengono creati alcuni punti

intermedi disposti attorno alla Red Zone e tali punti vengono passati come waypoint nella richiesta alle Directions API.

Gli otto waypoint sono distribuiti uniformemente lungo una circonferenza, con un angolo di 45 gradi tra un punto e il successivo. La distanza dal centro della Red Zone è stata fissata a circa 2.6 volte il raggio dell'area di pericolo, in modo da aumentare la probabilità che il nuovo percorso aggiri completamente la zona critica.

Per ciascun insieme di waypoint viene effettuata una nuova richiesta alle Directions API. Tutti i percorsi ottenuti vengono successivamente valutati con lo stesso algoritmo di conteggio delle intersezioni e viene infine selezionato quello che attraversa il minor numero di Red Zone. Sebbene questa tecnica non garantisca sempre la soluzione ottima, nei test effettuati ha prodotto percorsi sensibilmente più sicuri rispetto a quelli restituiti direttamente dal servizio di Google.

Funzione Seguimi a casa

Un'altra funzionalità rilevante dell'applicazione Android è Seguimi a casa, pensata per permettere all'utente di condividere in tempo reale la propria posizione con i contatti di emergenza.

La funzione viene avviata dalla Home tramite il pulsante Seguimi a casa. Alla pressione del pulsante, `HomeFragment` naviga verso `RedZonesFragment` passando il parametro `followMeMode = true`. In questo modo lo stesso fragment utilizzato per la mappa e per le Red Zone viene riutilizzato anche per la gestione del tracciamento live.

All'apertura della schermata viene mostrato un dialog che richiede all'utente di inserire la destinazione del tragitto. Una volta confermata la destinazione, viene creata una nuova sessione nella collection `liveLocations` di Firestore.

```
1 private fun startFollowMe(destination: String) {
2
3     val session = SessionManager(requireContext())
4
5     val dto = FollowMeSessionDto(
6         userId = session.getUserId(),
7         userName = session.getName(),
8         destination = destination
9     )
10
11     followMeRepository.stopActiveSessionsForUser(session.getUserId()) {
12         followMeRepository.createSession(
13             session = dto,
14             onSuccess = { sessionId ->
15                 currentLiveSessionId = sessionId
16
17                 val followMeLink = buildFollowMeLink(sessionId)
18
19                 Log.d("SHIELDUP_FOLLOWME", "NUOVO LINK FOLLOWME: $followMeLink")
20
21                 followMeManager.startTracking(sessionId)
22                 sendFollowMeLinkToEmergencyContacts(followMeLink)
23                 showStopFollowMeButton()
24
25                 requireActivity().runOnUiThread {
26                     Toast.makeText(
27                         requireContext(),
28                         "Condivisione posizione avviata",
29                         Toast.LENGTH_LONG
30                     ).show()
31
32                     calculateSafeRoute(destination)
33                 }
34             },
35             onError = {
36                 requireActivity().runOnUiThread {
37                     Toast.makeText(
38                         requireContext(),
39                         "Errore avvio Follow Me",
40                         Toast.LENGTH_LONG
41                     ).show()
42                 }
43             }
44         )
45     }
46 }
```

Listing 8: Creazione della sessione Follow Me

La sessione contiene le informazioni principali necessarie al tracciamento: identificativo dell'utente, nome dell'utente, destinazione, coordinate geografiche correnti, stato della sessione e timestamp di aggiornamento.

```
1 data class FollowMeSessionDto(  
2     val id: String = "",  
3     val userId: String = "",  
4     val userName: String = "",  
5     val destination: String = "",  
6     val latitude: Double = 0.0,  
7     val longitude: Double = 0.0,  
8     val active: Boolean = true,  
9     val startedAt: Long = System.currentTimeMillis(),  
10    val updatedAt: Long = System.currentTimeMillis()  
11 )
```

Listing 9: DTO della sessione Follow Me

Prima di creare una nuova sessione, l'applicazione disattiva eventuali sessioni live rimaste aperte per lo stesso utente. Questa scelta evita la presenza di più tracciamenti attivi contemporaneamente.

```
1 db.collection("liveLocations")  
2     .whereEqualTo("userId", userId)  
3     .whereEqualTo("active", true)  
4     .get()
```

Listing 10: Disattivazione delle sessioni precedenti

Una volta creata la sessione, il sistema costruisce un link condivisibile contenente l'identificativo del documento Firestore:

```
1 private fun buildFollowMeLink(sessionId: String): String {  
2     return "https://shieldup-925d1.web.app/followme.html?id=$sessionId"  
3 }
```

Listing 11: Generazione del link condivisibile

Il link viene inviato tramite SMS a tutti i contatti di emergenza salvati dall'utente. Per gestire messaggi più lunghi del limite standard degli SMS, viene utilizzato `sendMultipartTextMessage`.

```
1 val smsManager = SmsManager.getDefault()  
2 val parts = smsManager.divideMessage(message)  
3  
4 smsManager.sendMultipartTextMessage(  
5     phone,  
6     null,  
7     parts,  
8     null,  
9     null  
10 )
```

Listing 12: Invio del link tramite SMS

Il tracciamento della posizione è gestito dalla classe `FollowMeManager`. Questa classe utilizza un `Handler` per eseguire periodicamente un `Runnable`. Ogni cinque secondi viene recuperata l'ultima posizione disponibile tramite `FusedLocationProviderClient` e vengono aggiornati i campi `latitude`, `longitude` e `updatedAt` nel documento Firestore.

```
1 private val updateRunnable = object : Runnable {  
2     override fun run() {  
3         if (!running) return  
4     }
```

```

5         updateCurrentLocation()
6
7         handler.postDelayed(this, 5_000L)
8     }
9 }

```

Listing 13: Aggiornamento periodico della posizione

```

1 db.collection("liveLocations")
2   .document(sessionId)
3   .update(
4     mapOf(
5       "latitude" to latitude,
6       "longitude" to longitude,
7       "updatedAt" to System.currentTimeMillis()
8     )
9   )

```

Listing 14: Scrittura delle coordinate su Firestore

Dal punto di vista dell'utente, durante la condivisione viene aggiunto dinamicamente un pulsante rosso Termina condivisione. Alla pressione del pulsante, il tracciamento viene interrotto e il documento Firestore viene aggiornato impostando `active = false`.

```

1 db.collection("liveLocations")
2   .document(sessionId)
3   .update(
4     mapOf(
5       "active" to false,
6       "updatedAt" to System.currentTimeMillis()
7     )
8   )

```

Listing 15: Terminazione della sessione

La sessione viene inoltre terminata automaticamente nel metodo `onDestroyView()` di `RedZonesFragment`. Questo comportamento evita di lasciare sessioni attive quando l'utente abbandona la schermata, anche se rappresenta una limitazione nel caso in cui si voglia mantenere il tracciamento attivo in background.

Per rendere il link realmente apribile da un contatto esterno, è stata realizzata anche una piccola web app pubblicata tramite Firebase Hosting. Il deploy utilizza la cartella `public`, configurata nel file `firebase.json`. All'interno di questa cartella sono presenti i file `followme.html` e `followme.js`.

La pagina web riceve l'identificativo della sessione direttamente dalla query string dell'URL. Ad esempio, nel link:

`https://shieldup-925d1.web.app/followme.html?id=SESSION_ID`

il parametro `id` viene letto dallo script JavaScript e utilizzato per aprire un listener sul documento corrispondente nella collection `liveLocations`.

La pagina utilizza Firebase Web SDK per leggere Firestore e Google Maps JavaScript API per visualizzare la posizione su una mappa. Il listener in tempo reale consente di aggiornare automaticamente marker e posizione ogni volta che l'app Android scrive nuove coordinate sul database.

In questo modo il contatto di emergenza non deve installare l'applicazione: è sufficiente aprire il link ricevuto via SMS per monitorare il percorso dell'utente tramite browser.

La soluzione realizzata è funzionante e consente un tracciamento in tempo reale, ma presenta alcune limitazioni. Il tracciamento è legato al ciclo di vita di `RedZonesFragment`; di conseguenza, se l'utente chiude la schermata o l'app viene sospesa, la sessione viene interrotta. Una soluzione più robusta prevederebbe l'utilizzo di un `Foreground Service`, così da mantenere il tracciamento attivo anche con l'app in background.

Un'altra limitazione riguarda la sicurezza della parte web. La pagina pubblicata su Firebase Hosting contiene la configurazione Firebase e la chiave Google Maps JavaScript API. In un'applicazione reale sarebbe necessario applicare restrizioni alle API key e configurare regole Firestore più rigide, in modo da consentire l'accesso solo alle sessioni autorizzate.

Supporto psicologico

La sezione di supporto psicologico consente all'utente standard di richiedere assistenza attraverso due modalità: una richiesta urgente oppure la prenotazione di uno slot disponibile.

Nel caso della richiesta urgente, l'applicazione crea un nuovo documento nella collection `urgentPsychologicalRequests`, impostando lo stato iniziale della richiesta a `WAITING`. Tale richiesta viene poi visualizzata nella dashboard degli psicologi.

Nel caso della prenotazione ordinaria, l'utente visualizza gli slot futuri disponibili nella collection `availabilities`. Quando viene selezionato uno slot, il documento corrispondente viene aggiornato impostando il campo `booked` a `true` e associando lo slot all'utente che ha effettuato la prenotazione.

Questa distinzione permette di gestire sia situazioni immediate, nelle quali l'utente richiede un contatto rapido, sia situazioni programmate, nelle quali viene fissato un consulto in un momento successivo.

Dashboard psicologo

Gli utenti registrati con ruolo `PSYCHOLOGIST` accedono ad una dashboard dedicata. Questa schermata consente allo psicologo di gestire le proprie disponibilità e di visualizzare le richieste urgenti inviate dagli utenti.

La creazione di una disponibilità avviene tramite un dialog nel quale lo psicologo seleziona data e ora. Prima del salvataggio viene verificato che lo slot inserito sia collocato nel futuro. Le disponibilità vengono salvate nella collection `availabilities` e successivamente mostrate agli utenti standard nella sezione Supporto.

Quando lo psicologo accetta una richiesta urgente, il sistema aggiorna il documento nella collection `urgentPsychologicalRequests`, impostando lo stato a `ACCEPTED`. Contestualmente viene creata una nuova chat nella collection `chats`, così da permettere la comunicazione privata tra psicologo e utente.

Sistema di chat

La chat è stata implementata attraverso due schermate: una lista delle conversazioni e una stanza di messaggistica.

La collection `chats` contiene le informazioni principali della conversazione, tra cui utente, psicologo, ultimo messaggio e data di creazione. I messaggi veri e propri sono invece salvati nella sotto-collezione `chats/{chatId}/messages`.

Quando un utente o uno psicologo invia un messaggio, viene creato un nuovo documento nella sotto-collezione dei messaggi e viene aggiornato anche il documento principale della chat con il testo e il timestamp dell'ultimo messaggio.

La chat della versione Android utilizza query singole verso Firestore e non listener real-time. Di conseguenza i messaggi non vengono aggiornati automaticamente in tempo reale, ma vengono ricaricati all'apertura della stanza o dopo l'invio di un nuovo messaggio. Questa scelta ha semplificato l'implementazione, pur rappresentando una possibile area di miglioramento.

Chiamata simulata

Un'altra funzionalità particolare sviluppata durante il progetto è la “Chiamata simulata”, pensata per aiutare l'utente ad uscire rapidamente da situazioni di disagio o potenziale pericolo. L'idea consiste nel simulare la ricezione di una telefonata apparentemente reale, permettendo all'utente di utilizzare come pretesto la necessità di rispondere alla chiamata.

Dal punto di vista dell'esperienza utente, il funzionamento è molto semplice. Dopo aver selezionato la funzione dalla schermata principale, l'utente sceglie il nome del chiamante e il tempo di attesa desiderato (10, 30 oppure 60 secondi). Trascorso il tempo impostato, compare una schermata che riproduce l'interfaccia di una normale chiamata Android, completa del nome del chiamante e dei pulsanti di risposta o rifiuto.

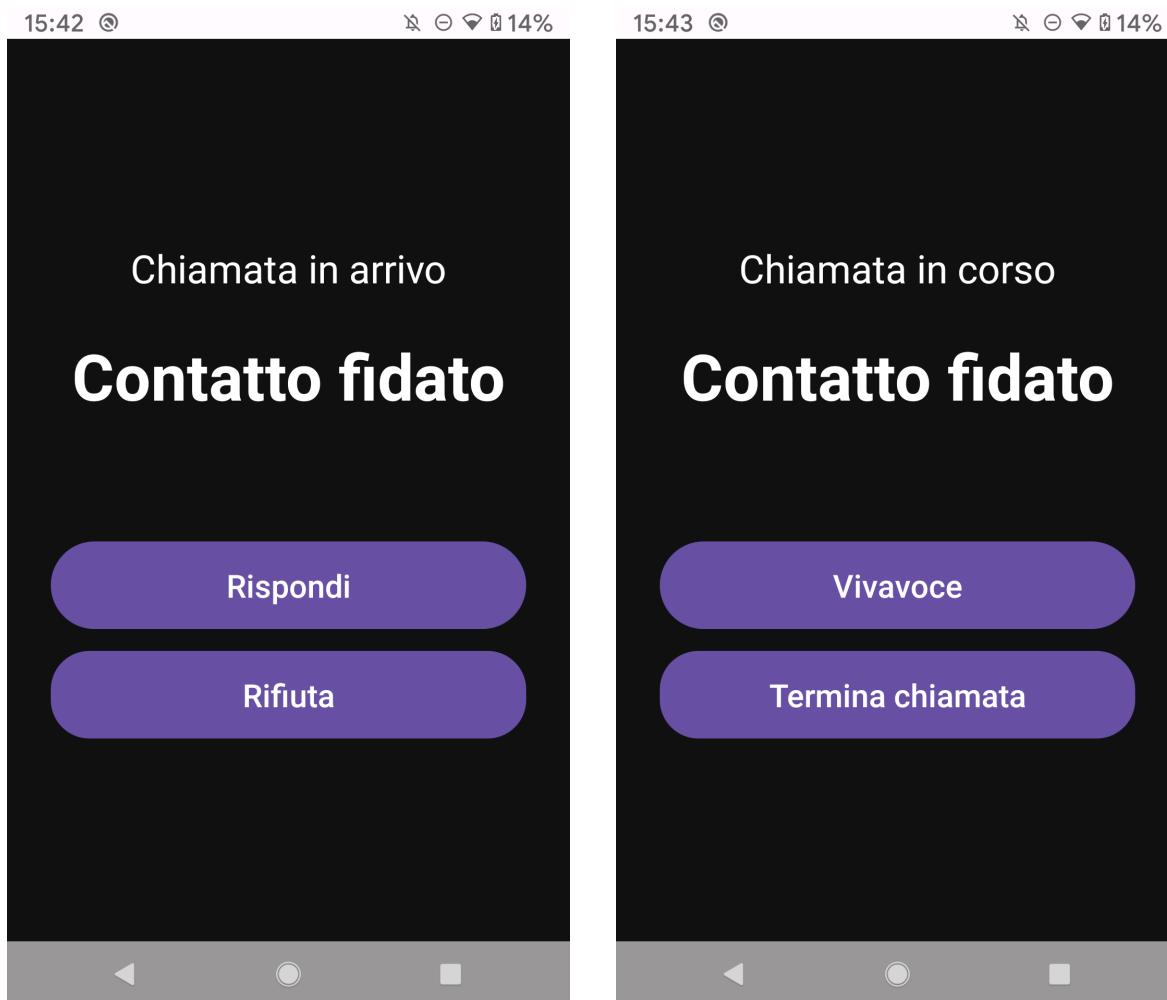


Figure 1: *Confronto tra le due schermate.*

Durante la fase di progettazione avevamo inizialmente previsto un'implementazione molto più simile al comportamento delle normali chiamate telefoniche del sistema operativo Android.

L'idea era quella di utilizzare una notifica Full Screen associata ad un **BroadcastReceiver**. Allo scadere del timer il receiver avrebbe ricevuto un intent programmato e avrebbe generato una notifica ad alta priorità capace di aprire automaticamente la schermata della chiamata simulata anche con il telefono bloccato o mentre l'utente stava utilizzando altre applicazioni.

Per questo motivo sono state sviluppate le classi:

- **FakeCallNotification**, incaricata della creazione del Notification Channel e della visualizzazione della notifica Full Screen;
- **FakeCallReceiver**, incaricato di ricevere il Broadcast e avviare la schermata della chiamata.

Questa soluzione, tuttavia, ha evidenziato numerosi problemi durante lo sviluppo.

Le versioni più recenti di Android introducono infatti numerose limitazioni sulla possibilità di avviare Activity in background. Inoltre, l'utilizzo delle notifiche Full Screen richiede particolari autorizzazioni e viene consentito solo in specifici casi d'uso (ad esempio applicazioni telefoniche, sistemi di allarme o chiamate VoIP). Durante i test sono emersi comportamenti differenti a seconda della versione del sistema operativo e del produttore del dispositivo, rendendo il comportamento poco affidabile.

Un ulteriore problema riguardava la gestione del timer. Utilizzando esclusivamente componenti di sistema, il timer avrebbe dovuto continuare a funzionare anche nel caso in cui l'applicazione fosse stata chiusa o il dispositivo fosse entrato in modalità standby. La realizzazione di questo comportamento avrebbe richiesto l'introduzione di ulteriori componenti, come **AlarmManager** oppure un **Foreground Service**, aumentando notevolmente la complessità del progetto.

Considerando il tempo disponibile, abbiamo deciso di adottare una soluzione più semplice ma completamente funzionante.

La programmazione della chiamata viene gestita in **HomeFragment**. Quando l'utente seleziona la funzione, viene mostrato un **AlertDialog** che consente di inserire il nome del chiamante e scegliere il ritardo tra 10, 30 e 60 secondi.

```
1 val delayMillis = when (selectedPosition) {  
2     0 -> 10_000L  
3     1 -> 30_000L  
4     else -> 60_000L  
5 }
```

Listing 16: Scelta del ritardo della chiamata simulata

Dopo la conferma, il metodo **scheduleFakeCall** utilizza un **Handler** associato al thread principale per avviare la chiamata simulata dopo il ritardo selezionato.

```
1 private fun scheduleFakeCall(callerName: String, delayMillis: Long) {  
2     Handler(Looper.getMainLooper()).postDelayed({  
3         if (!isAdded) return@postDelayed  
4  
5         val intent = Intent(requireContext(), FakeCallActivity::class.java).apply {  
6             putExtra("callerName", callerName)  
7         }  
8  
9         startActivity(intent)  
10    }, delayMillis)  
11 }
```

Listing 17: Programmazione della chiamata simulata

La schermata vera e propria della chiamata è invece gestita da `FakeCallActivity`. Questa riceve tramite `Intent` il nome del chiamante e mostra una schermata a tutto schermo con pulsanti per rispondere o rifiutare la chiamata.

0.3.5 Testing

Durante lo sviluppo dell'applicazione sono stati effettuati numerosi test funzionali al fine di verificare il corretto comportamento delle diverse funzionalità implementate. Non essendo previsto un framework di testing automatico, le verifiche sono state eseguite manualmente utilizzando dispositivi Android reali ed emulatori Android Studio.

Per ciascuna funzionalità è stato verificato sia il corretto comportamento in condizioni normali sia la gestione delle situazioni di errore, come la mancanza di connessione ad Internet, la negazione dei permessi di sistema o l'inserimento di dati non validi.

Test dell'autenticazione

La prima serie di test ha riguardato il sistema di autenticazione.

Sono stati verificati:

- registrazione di nuovi utenti;
- login con credenziali corrette;
- login con password errata;
- tentativo di registrazione con email già presente;
- mantenimento della sessione dopo il riavvio dell'applicazione;
- corretto reindirizzamento in base al ruolo (utente standard o psicologo).

I test hanno confermato il corretto funzionamento della memorizzazione della sessione tramite `SharedPreferences` e la corretta distinzione tra i due profili previsti dall'applicazione.

Test delle Red Zone

Per la funzionalità delle Red Zone sono stati effettuati diversi test relativi alla creazione, visualizzazione e scadenza automatica delle segnalazioni.

In particolare è stato verificato che:

- la segnalazione venga salvata correttamente su Firestore;
- il marker venga visualizzato sulla mappa;
- il cerchio di sicurezza abbia il raggio previsto;
- le segnalazioni vengano rimosse automaticamente dopo un'ora;
- la lista delle Red Zone mostri separatamente segnalazioni attive e recenti.

Sono stati inoltre effettuati test con più dispositivi per verificare la sincronizzazione delle segnalazioni tramite Firestore.

Test del percorso sicuro

Il percorso sicuro rappresenta una delle funzionalità più complesse dell'applicazione ed ha richiesto numerosi test.

Le prove hanno riguardato:

- calcolo del percorso senza Red Zone;
- presenza di una singola Red Zone lungo il tragitto;
- presenza contemporanea di più aree di pericolo;
- generazione dei waypoint alternativi;
- corretto disegno della polilinea finale.

Sono stati inoltre confrontati i percorsi restituiti dalle Google Directions API con quelli ottenuti dopo l'applicazione dell'algoritmo di deviazione sviluppato durante il progetto, verificando che il sistema privilegiasse sempre il percorso con il minor numero di intersezioni con le aree considerate pericolose.

Test della funzione “Seguimi a casa”

Per la funzionalità di condivisione della posizione sono stati utilizzati due dispositivi fisici.

Il primo dispositivo eseguiva l'applicazione Android, mentre il secondo apriva il link ricevuto tramite SMS nel browser.

Sono stati verificati:

- creazione della sessione su Firestore;
- invio corretto del link ai contatti di emergenza;
- aggiornamento delle coordinate GPS ogni cinque secondi;
- aggiornamento automatico della mappa tramite listener Firestore;
- corretta terminazione della sessione.

Le prove hanno evidenziato che il sistema funziona correttamente finché il fragment rimane attivo, mentre il tracciamento viene interrotto nel momento in cui l'utente abbandona la schermata, come discusso nei capitoli precedenti.

Test del supporto psicologico

Per il modulo di supporto psicologico sono stati utilizzati due account distinti: uno con ruolo STANDARD e uno con ruolo PSYCHOLOGIST.

Sono stati verificati:

- inserimento delle disponibilità dello psicologo;
- visualizzazione degli slot da parte dell'utente;

- prenotazione degli appuntamenti;
- invio delle richieste urgenti;
- accettazione delle richieste da parte dello psicologo;
- creazione automatica della chat.

Tutti i dati sono stati sincronizzati correttamente attraverso Cloud Firestore.

Test della chat

Per la chat privata sono stati effettuati test di invio e ricezione dei messaggi tra due utenti appartenenti ai ruoli previsti.

È stato verificato il corretto aggiornamento della collection **messages** e del campo relativo all'ultimo messaggio nella collection **chats**.

Durante i test è emersa la principale limitazione dell'implementazione: la mancanza di un listener real-time. Di conseguenza, i nuovi messaggi risultano visibili solamente dopo un nuovo caricamento della conversazione oppure dopo l'invio di un ulteriore messaggio.

Test della chiamata simulata

Sono stati effettuati test utilizzando tutti i ritardi disponibili (10, 30 e 60 secondi) verificando il corretto avvio della schermata della chiamata simulata.

È stato inoltre verificato il comportamento dei pulsanti di risposta e chiusura della chiamata e il corretto aggiornamento dell'interfaccia durante la simulazione.

0.4 Flutter

La seconda implementazione di ShieldUp è stata sviluppata utilizzando **Flutter** e il linguaggio **Dart**. A differenza della versione Android nativa, in questo caso l'applicazione è stata progettata come progetto multiplatforma, mantenendo però lo stesso database Firebase utilizzato dalla versione Kotlin.

La versione Flutter condivide infatti le principali collection Firestore già descritte nel capitolo Android, in particolare `users`, `redZones` e la sotto-collezione `emergencyContacts`. Questa scelta ha permesso di mantenere una struttura dati comune tra le due implementazioni, pur utilizzando tecnologie differenti per la realizzazione dell'interfaccia e della logica applicativa.

Rispetto alla versione Android nativa, l'implementazione Flutter è stata sviluppata come versione semplificata, concentrandosi sulle funzionalità principali: autenticazione, gestione del profilo, contatti di emergenza, chiamata ai soccorsi, mappa con segnalazioni e chiamata simulata. Alcune funzionalità avanzate presenti nella versione Android, come il percorso sicuro, il tracciamento “Seguimi a casa” e la gestione completa del supporto psicologico, non sono state replicate nella versione Flutter.

0.4.1 Architettura

L'applicazione Flutter è organizzata seguendo una struttura modulare, nella quale la logica di accesso ai dati è separata dall'interfaccia utente. A differenza della versione Android nativa, basata su Activity e Fragment, Flutter realizza l'intera interfaccia mediante widget, consentendo di utilizzare lo stesso codice su più piattaforme.

Il punto di ingresso dell'applicazione è rappresentato dal file `main.dart`, che inizializza Firebase tramite `Firebase.initializeApp()` e configura il tema grafico globale dell'applicazione prima di avviare il widget principale.

La navigazione tra le schermate viene gestita attraverso il componente `Navigator` di Flutter. In particolare, dopo il login, l'utente viene indirizzato verso un'interfaccia differente in base al ruolo associato al proprio account. Gli utenti standard accedono alla schermata `UserHomeScreen`, mentre gli psicologi vengono reindirizzati a `PsychologistHomeScreen`. Le operazioni di login e logout utilizzano principalmente i metodi `Navigator.pushReplacement()` e `Navigator.pushAndRemoveUntil()`, evitando che l'utente possa tornare alle schermate precedenti tramite il pulsante Indietro.

Il progetto è suddiviso nei seguenti package principali:

- **data**: contiene i model dell'applicazione e la classe `FirebaseRepository`, responsabile della comunicazione con Cloud Firestore;
- **screens**: contiene tutte le schermate dell'applicazione, tra cui login, registrazione, home utente, dashboard dello psicologo, mappa, profilo, contatti di emergenza, chiamata simulata e procedura di emergenza;
- la cartella principale **lib**: contiene `main.dart`, punto di ingresso dell'applicazione, e il file di configurazione `firebase_options.dart` generato automaticamente da Firebase.

I principali widget che compongono l'interfaccia sono organizzati in schermate indipendenti, ciascuna responsabile di una specifica funzionalità. Le liste dinamiche vengono costruite mediante widget come `ListView.builder`, mentre la navigazione inferiore è realizzata tramite `NavigationBar`.

L'accesso ai dati è centralizzato nella classe `FirestoreRepository`, che espone i metodi necessari per registrazione, autenticazione, gestione dei contatti di emergenza e delle Red Zone. La comunicazione con il database avviene tramite Cloud Firestore, utilizzando sia operazioni CRUD tradizionali sia stream in tempo reale per aggiornare automaticamente alcune schermate, come la lista dei contatti e quella delle segnalazioni.

Per le funzionalità geografiche vengono utilizzati il pacchetto `google_maps_flutter` per la visualizzazione della mappa e `geolocator` per ottenere la posizione corrente del dispositivo. L'avvio della chiamata ai servizi di emergenza viene invece gestito tramite il pacchetto `url_launcher`, che consente di aprire il dialer del sistema operativo con il numero 112 già compilato.

0.4.2 Database

Anche la versione Flutter utilizza **Firestore** come sistema di persistenza dei dati. Le due implementazioni condividono infatti lo stesso database, permettendo ad entrambe di accedere alle medesime informazioni senza richiedere strutture dati differenti.

Le collection utilizzate sono quindi le stesse già descritte nel capitolo dedicato all'implementazione Android. In particolare, la versione Flutter utilizza:

- **users**, contenente gli account degli utenti e degli psicologi;
- **users/{userId}/emergencyContacts**, che memorizza i contatti di emergenza associati ad ogni utente;
- **redZones**, contenente le segnalazioni di pericolo visualizzate sulla mappa.

L'accesso ai dati è centralizzato nella classe **FirestoreRepository**, che raccoglie tutti i metodi necessari per interagire con Cloud Firestore. Questa classe si occupa delle operazioni di autenticazione, registrazione degli utenti, aggiornamento del profilo, gestione dei contatti di emergenza e inserimento delle Red Zone.

A differenza della versione Android, alcune schermate sfruttano gli stream messi a disposizione dal pacchetto **cloud_firestore**. In particolare, la lista dei contatti di emergenza e quella delle segnalazioni vengono aggiornate automaticamente quando il contenuto del database cambia, senza richiedere un aggiornamento manuale dell'interfaccia.

Come nella versione Android, anche in questo progetto l'autenticazione è stata implementata direttamente mediante query sulla collection **users**, senza utilizzare Firebase Authentication. Le informazioni dell'utente vengono quindi recuperate confrontando l'indirizzo e-mail e la password memorizzati nel database.

0.4.3 UI

L'interfaccia grafica della versione Flutter è stata realizzata interamente mediante widget del framework, senza utilizzare file di layout XML come avviene nello sviluppo Android nativo. Tutte le schermate sono costruite componendo widget Material, mantenendo uno stile grafico coerente con la versione Android e privilegiando semplicità e immediatezza d'uso.

La navigazione è organizzata mediante il widget **NavigationBar**, che permette di passare rapidamente tra le diverse sezioni dell'applicazione. Anche in questo caso vengono mostrate schermate differenti in base al ruolo dell'utente autenticato.

Tutti gli screenshot riportati in questa sezione sono stati acquisiti durante le attività di testing della versione Flutter eseguita su dispositivo Apple iPhone 15.

Home

La schermata Home rappresenta il punto di accesso principale per gli utenti standard. Essa mostra un messaggio di benvenuto personalizzato e tre pulsanti che consentono di accedere rapidamente alle funzionalità principali dell'applicazione:

- chiamata ai soccorsi;
- segnalazione di un pericolo;
- chiamata simulata.

Le funzionalità Seguimi a casa e Percorso sicuro, presenti nella versione Android, non sono state implementate nella versione Flutter. L'interfaccia risulta quindi più essenziale, mantenendo però l'accesso immediato alle principali funzioni di emergenza.

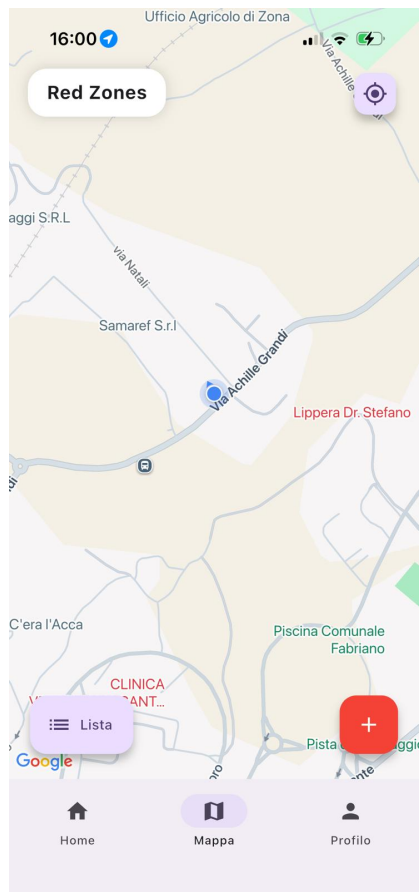


Mappa

La schermata della mappa integra il pacchetto Google Maps Flutter e permette di visualizzare le Red Zone salvate nel database Firestore.

Ogni segnalazione viene rappresentata mediante un marker e da un'area circolare evidenziata in rosso. L'utente può inoltre aggiungere nuove segnalazioni, consultare l'elenco delle Red Zone attive e centrare rapidamente la mappa sulla propria posizione corrente.

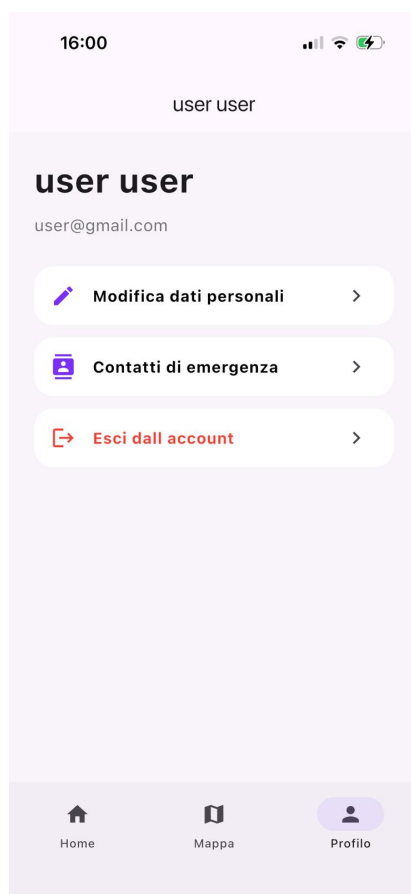
L'inserimento di una nuova segnalazione avviene tramite una dialog windows nella quale vengono richiesti titolo, descrizione e modalità di acquisizione della posizione.



Profilo

La schermata Profilo consente all'utente di visualizzare le proprie informazioni personali e di accedere alle principali operazioni di gestione dell'account.

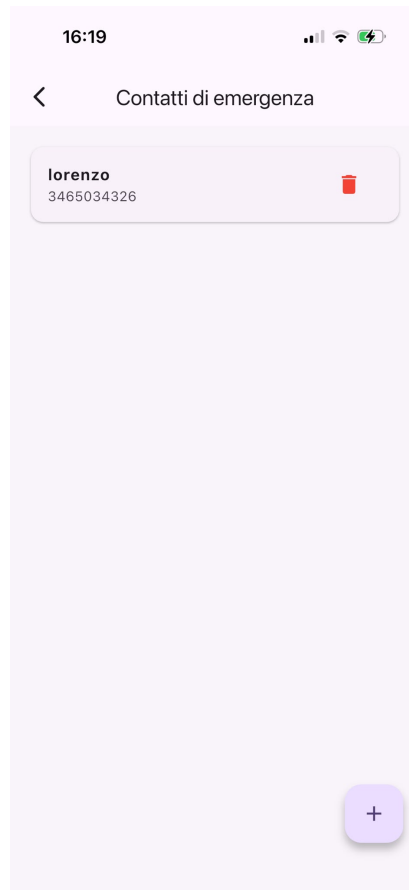
Da questa schermata è possibile modificare nome, cognome ed indirizzo e-mail, accedere alla lista dei contatti di emergenza ed effettuare il logout. Per gli utenti registrati come psicologi la voce relativa ai contatti di emergenza non viene mostrata.



Contatti di emergenza

La gestione dei contatti di emergenza è dedicata agli utenti standard e permette di memorizzare numeri telefonici fidati da utilizzare in caso di necessità.

La schermata mostra l'elenco dei contatti salvati e consente sia l'aggiunta di nuovi numeri sia l'eliminazione di quelli esistenti. I dati vengono sincronizzati automaticamente con Cloud Firestore.



Chiamata simulata

La versione Flutter implementa una simulazione completa della chiamata telefonica composta da tre fasi: configurazione della chiamata, ricezione della chiamata e conversazione simulata.

L'utente può scegliere il nome del chiamante e il ritardo desiderato. Trascorso il tempo selezionato viene mostrata una schermata che riproduce una chiamata in arrivo, dalla quale è possibile rispondere oppure rifiutare la telefonata.

Nel caso in cui venga accettata la chiamata, l'interfaccia passa automaticamente alla schermata della conversazione simulata, mostrando un contatore della durata della chiamata e alcuni controlli grafici analoghi a quelli presenti nelle applicazioni telefoniche.

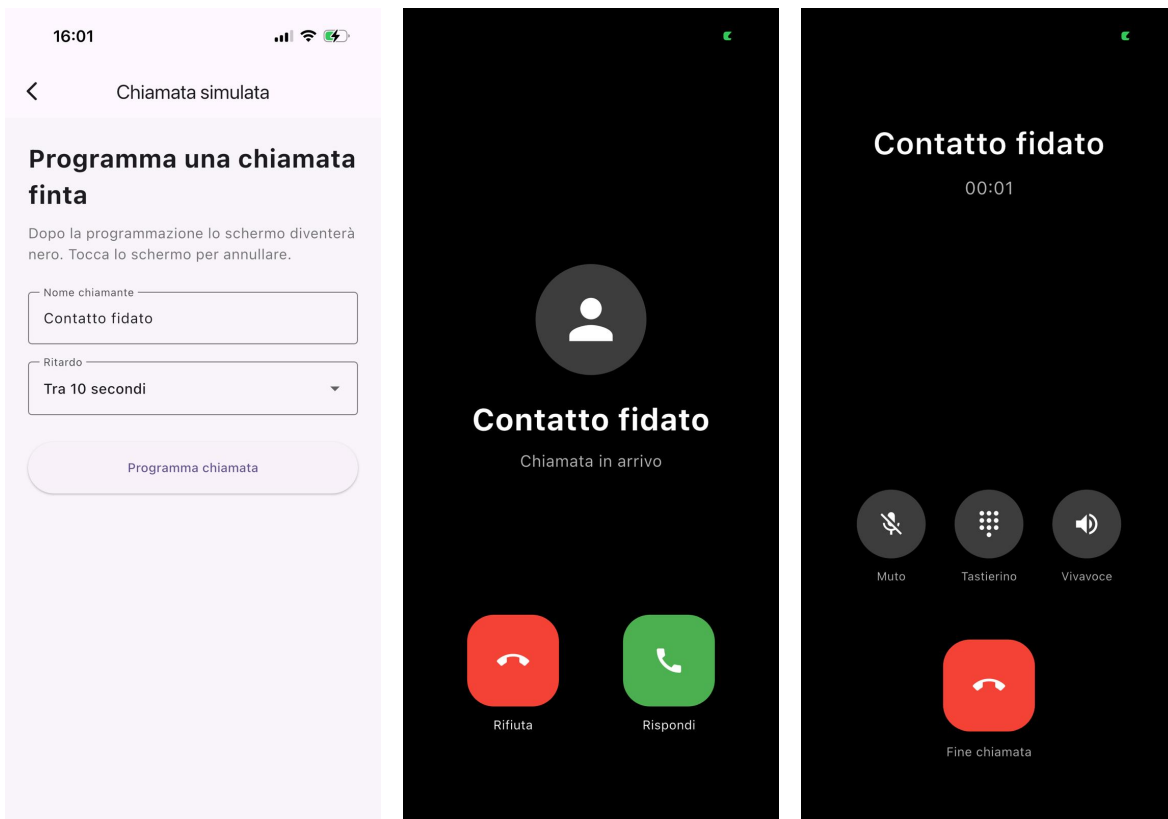


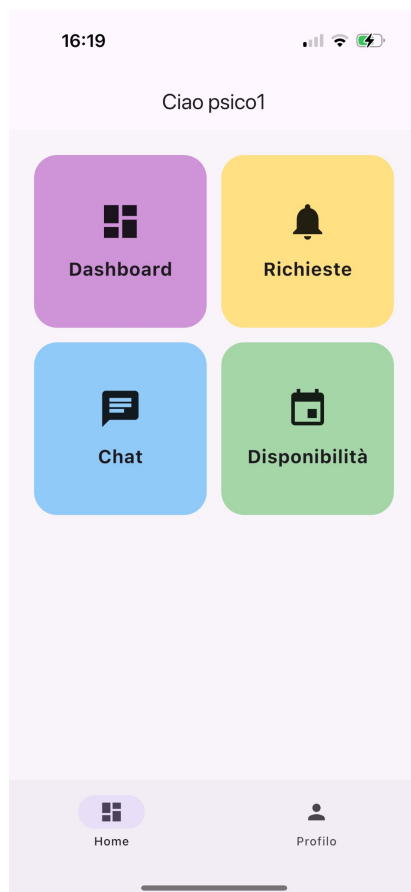
Figure 2: *Le tre fasi della chiamata simulata nella versione Flutter.*

Dashboard dello psicologo

Gli utenti registrati come psicologi visualizzano una schermata iniziale differente rispetto agli utenti standard.

La dashboard mostra quattro sezioni dedicate a Dashboard, Richieste, Chat e Disponibilità. Nella versione Flutter tali elementi hanno lo scopo di rappresentare la struttura dell'interfaccia prevista per una futura estensione dell'applicazione e non implementano la logica completa presente nella versione Android.

La schermata permette comunque allo psicologo di accedere al proprio profilo e di effettuare il logout.



0.4.4 Sviluppo

Lo sviluppo della versione Flutter ha seguito gli stessi requisiti funzionali definiti per l'applicazione Android, adattandoli però al framework Flutter. Le diverse funzionalità sono state implementate mediante widget indipendenti, mentre l'accesso ai dati è stato centralizzato nella classe `FirebaseRepository`, evitando di duplicare la logica di comunicazione con Cloud Firestore.

Nei paragrafi seguenti vengono descritte le principali funzionalità implementate.

Sistema di autenticazione

Anche nella versione Flutter il sistema di autenticazione è stato realizzato senza utilizzare Firebase Authentication. Le credenziali vengono memorizzate direttamente nella collection `users` di Cloud Firestore e vengono verificate mediante query eseguite dal repository.

Durante la registrazione vengono richiesti nome, cognome, indirizzo e-mail, password e ruolo dell'account. Prima del salvataggio vengono effettuati alcuni controlli sui dati inseriti, verificando che tutti i campi siano compilati e che l'indirizzo e-mail non sia già presente nel database.

Listing 18: Creazione del nuovo utente su Firestore

```
1 await users.add({  
2   "name": name,  
3   "surname": surname,  
4   "email": email.toLowerCase(),  
5   "password": password,  
6   "role": role,  
7   "createdAt": DateTime.now().millisecondsSinceEpoch,  
8 });
```


L'autenticazione viene effettuata tramite una query diretta sulla collection `users`. Prima della ricerca l'indirizzo e-mail viene normalizzato convertendolo in minuscolo ed eliminando eventuali spazi iniziali e finali.

Listing 19: Login tramite Firestore

```
1 Future<AppUser> loginUser({  
2     required String email,  
3     required String password,  
4 }) async {  
5     final normalizedEmail = email.trim().toLowerCase();  
6     final cleanPassword = password.trim();  
7  
8     final snapshot = await _db  
9         .collection('users')  
10        .where('email', isEqualTo: normalizedEmail)  
11        .limit(1)  
12        .get();  
13  
14    if (snapshot.docs.isEmpty) {  
15        throw Exception('Utente non trovato');  
16    }  
17  
18    final data = snapshot.docs.first.data();  
19  
20    if ((data['password'] ?? '') != cleanPassword) {  
21        throw Exception('Password non corretta');  
22    }  
23  
24    return AppUser.fromFirestore(snapshot.docs.first.id, data);  
25 }
```

Il metodo effettua una ricerca limitata ad un solo documento tramite `limit(1)`. Se non viene trovato alcun utente viene generata un'eccezione, mentre in caso di successo la password inserita viene confrontata direttamente con quella presente nel database. Se il controllo va a buon fine, il documento Firestore viene convertito in un oggetto `AppUser`, che verrà poi utilizzato dall'applicazione.

Questa soluzione è stata adottata per semplicità implementativa. In un'applicazione destinata alla produzione sarebbe preferibile utilizzare Firebase Authentication oppure salvare esclusivamente password cifrate mediante algoritmi di hashing.

Gestione del profilo

La schermata del profilo permette di visualizzare e modificare le informazioni personali dell'utente.

L'aggiornamento dei dati avviene tramite una dialog window contenente i campi nome, cognome ed e-mail. Prima di salvare le modifiche viene verificato che il nuovo indirizzo e-mail non appartenga già ad un altro account presente nel database.

Una volta completata la verifica, il documento dell'utente viene aggiornato su Cloud Firestore e l'interfaccia viene aggiornata automaticamente mostrando i nuovi dati.

Gestione dei contatti di emergenza

I contatti di emergenza vengono salvati all'interno della sotto-collezione dedicata di ciascun utente.

La schermata utilizza uno **StreamBuilder**, che rimane costantemente in ascolto delle modifiche provenienti da Firestore. Ogni variazione, come l'aggiunta o l'eliminazione di un contatto, viene quindi visualizzata immediatamente senza richiedere un aggiornamento manuale della pagina.

Per la costruzione dinamica della lista viene utilizzato il widget **ListView.builder**, che genera automaticamente una scheda per ogni contatto presente nel database.

Durante l'inserimento vengono controllati il nome del contatto e la validità del numero telefonico, limitando l'inserimento ai soli numeri di telefono con lunghezza adeguata.

Procedura di emergenza

La procedura di emergenza è stata implementata in modo più semplice rispetto alla versione Android.

Alla pressione del pulsante dedicato viene mostrato un conto alla rovescia della durata di dieci secondi. Durante questo intervallo l'utente può annullare l'operazione.

Listing 20: Gestione del conto alla rovescia

```
1 void startCountdown() {  
2     timer = Timer.periodic(  
3         const Duration(seconds: 1),  
4         (timer) async {  
5             if (secondsLeft <= 1) {  
6                 timer.cancel();  
7  
8                 final uri = Uri(  
9                     scheme: 'tel',  
10                    path: '112',  
11                );  
12  
13                if (await canLaunchUrl(uri)) {  
14                    await launchUrl(uri);  
15                }  
16  
17                return;  
18            }  
19  
20            setState(() {  
21                secondsLeft--;  
22            });  
23        },  
24    );  
25 }
```

Il conto alla rovescia viene realizzato mediante un `Timer.periodic()`, che esegue una callback ogni secondo. Ad ogni esecuzione il valore di `secondsLeft` viene decrementato e l'interfaccia viene aggiornata tramite `setState()`. Quando il timer raggiunge lo zero, il ciclo viene interrotto e viene creato un URI con schema `tel`. Successivamente il pacchetto `url_launcher` verifica la possibilità di effettuare una chiamata e apre automaticamente il dialer del dispositivo con il numero 112 già compilato.

Diversamente dalla versione Android, questa implementazione non invia SMS ai contatti di emergenza né acquisisce automaticamente la posizione GPS dell'utente.

Gestione delle Red Zone

La schermata della mappa integra Google Maps Flutter e consente sia la consultazione sia l'inserimento di nuove segnalazioni.

L'utente può scegliere se utilizzare la posizione GPS corrente oppure il centro della mappa visualizzata. Prima di recuperare la posizione del dispositivo vengono verificati i permessi di geolocalizzazione.

Listing 21: Acquisizione della posizione corrente

```
1 Future<LatLng> getCurrentPositionOrMapCenter() async {
2   try {
3     LocationPermission permission =
4       await Geolocator.checkPermission();
5
6     if (permission == LocationPermission.denied) {
7       permission =
8         await Geolocator.requestPermission();
9     }
10
11    if (permission ==
12      LocationPermission.denied ||
13      permission ==
14        LocationPermission.deniedForever) {
15      return anconaCenter;
16    }
17
18    final position =
19      await Geolocator.getCurrentPosition(
20        desiredAccuracy:
21          LocationAccuracy.high,
22      );
23
24    return LatLng(
25      position.latitude,
26      position.longitude,
27    );
28  } catch (_) {
29    return anconaCenter;
30  }
31 }
```

Il metodo verifica innanzitutto che l'applicazione disponga dei permessi necessari. Se l'utente li nega, oppure si verifica un errore durante il recupero della posizione, viene restituita una coordinata predefinita corrispondente al centro della città di Ancona. In questo modo la mappa può comunque essere utilizzata anche senza l'accesso al GPS.

Le Red Zone vengono recuperate tramite uno stream Firestore.

Listing 22: Filtro temporale delle Red Zone

```
1 Stream<List<RedZone>> watchRedZones() {
2   return _db
3     .collection('redZones')
4     .orderBy('createdAt',
5       descending: true)
6     .snapshots()
7     .map((snapshot) {
8       final now =
9         DateTime.now()
10        .millisecondsSinceEpoch;
11
12       const durationMs =
13         60 * 60 * 1000;
14
15       return snapshot.docs
16         .map((doc) =>
17           RedZone.fromFirestore(
18             doc.id,
19             doc.data()))
20         .where((zone) =>
21           now - zone.createdAt <
22             durationMs)
23         .toList();
24     });
25 }
```

Lo stream converte ciascun documento in un oggetto `RedZone` e successivamente elimina tutte le segnalazioni più vecchie di un'ora, evitando di mostrare sulla mappa informazioni ormai non più attuali.

Le segnalazioni vengono poi trasformate nei componenti grafici visualizzati sulla mappa.

Listing 23: Creazione di marker e aree di rischio

```
1 Set<Marker> buildMarkers(
2   List<RedZone> zones) {
3   return zones.map((zone) {
4     return Marker(
5       markerId: MarkerId(zone.id),
6       position: LatLng(
7         zone.latitude,
8         zone.longitude,
9       ),
10    );
11  }).toSet();
12 }
13
14 Set<Circle> buildCircles(
15   List<RedZone> zones) {
16   return zones.map((zone) {
17     return Circle(
18       circleId: CircleId(zone.id),
19       center: LatLng(
20         zone.latitude,
21         zone.longitude,
22       ),
23       radius: 160,
24     );
25  }).toSet();
26 }
```

Per ogni segnalazione vengono creati sia un marker sia un'area circolare con raggio di 160 metri. I marker identificano il punto in cui è stata effettuata la segnalazione, mentre i cerchi rappresentano visivamente l'estensione dell'area considerata a rischio.

Chiamata simulata

La chiamata simulata è una delle funzionalità più complete della versione Flutter.

L'utente può scegliere il nome del chiamante e il tempo di attesa desiderato.

Listing 24: Pianificazione della chiamata simulata

```
1 void scheduleCall() {
2   callerName =
3     callerController.text.trim().isEmpty
4       ? 'Contatto fidato'
5       : callerController.text.trim();
6
7   waitingTimer?.cancel();
8   callTimer?.cancel();
9
10  setState(() {
11    waiting = true;
12    incomingCall = false;
13    inCall = false;
14  });
15
16  waitingTimer = Timer(
17    Duration(
18      seconds:
19        selectedDelaySeconds,
20    ),
21    () {
22      if (!mounted) return;
23
24      setState(() {
25        waiting = false;
26        incomingCall = true;
27      });
28    },
29  );
30 }
```

Il funzionamento è basato su tre variabili di stato: `waiting`, `incomingCall` e `inCall`. Inizialmente l'applicazione entra nello stato di attesa, mostrando uno schermo completamente nero. Allo scadere del timer viene modificato lo stato del widget e l'interfaccia viene ricostruita automaticamente mostrando la schermata della chiamata in arrivo.

Se l'utente accetta la chiamata viene visualizzata una schermata che simula una conversazione telefonica completa di cronometro e pulsanti grafici. Tutti questi controlli hanno una funzione esclusivamente dimostrativa e non interagiscono con l'hardware del dispositivo.

Dashboard dello psicologo

La versione Flutter prevede una schermata dedicata agli utenti registrati come psicologi.

La dashboard rappresenta una versione semplificata rispetto a quella sviluppata per Android e contiene le principali sezioni previste dall'applicazione, tra cui Dashboard, Richieste, Chat e Disponibilità.

Nella versione consegnata tali funzionalità hanno principalmente lo scopo di mostrare la struttura dell'interfaccia e costituiscono una base per eventuali sviluppi futuri. La schermata consente comunque allo psicologo di accedere al proprio profilo ed effettuare il logout.

0.4.5 Testing

Anche per la versione Flutter sono stati eseguiti numerosi test funzionali, con l'obiettivo di verificare il corretto comportamento delle principali funzionalità implementate. Le prove sono state effettuate principalmente su un iPhone 15, verificando sia i flussi principali sia la gestione delle situazioni di errore.

Test dell'autenticazione

Sono stati verificati:

- registrazione di nuovi utenti;
- login con credenziali corrette;
- login con password errata;
- registrazione con indirizzo e-mail già presente;
- corretto reindirizzamento dell'utente standard e dello psicologo.

Test delle Red Zone

Per la funzionalità della mappa sono stati verificati:

- recupero della posizione GPS;
- inserimento di una nuova segnalazione;
- sincronizzazione con Cloud Firestore;
- visualizzazione dei marker e delle aree rosse;
- aggiornamento automatico delle segnalazioni.

Test dei contatti di emergenza

Sono stati verificati:

- inserimento di un nuovo contatto;
- eliminazione di un contatto;
- sincronizzazione automatica tramite StreamBuilder.

Test della chiamata ai soccorsi

È stato verificato il corretto funzionamento del conto alla rovescia e l'apertura della proposta di chiamata per le nuove versioni di ios.

Test della chiamata simulata

Sono stati testati tutti i ritardi disponibili (10, 30 e 60 secondi), verificando il corretto passaggio tra la schermata di configurazione, la simulazione della chiamata in arrivo e la conversazione simulata.

0.5 Confronto tra le implementazioni

Durante il progetto sono state sviluppate due implementazioni della stessa applicazione: una versione nativa Android sviluppata in Kotlin e una versione multiplatforma sviluppata in Flutter.

Entrambe condividono lo stesso database Firebase Cloud Firestore e gli stessi requisiti di base, ma differiscono per tecnologie utilizzate, organizzazione del codice e livello di completezza delle funzionalità implementate.

Funzionalità	Android	Flutter
Autenticazione	✓	✓
Gestione profilo	✓	✓
Contatti di emergenza	✓	✓
Red Zone	✓	✓
Chiamata ai soccorsi	✓	✓
Chiamata simulata	✓	✓
Percorso sicuro	✓	—
Seguimi a casa	✓	—
Supporto psicologico completo	✓	—
Sistema di chat	✓	—
Dashboard psicologo	✓	Parziale

Table 13: *Confronto tra le implementazioni Android e Flutter.*

Dal confronto emerge come la versione Android rappresenti l'implementazione più completa del progetto, comprendendo anche funzionalità avanzate quali il percorso sicuro, il tracciamento della posizione in tempo reale e il sistema di supporto psicologico.

Nonostante alcune semplificazioni, entrambe le implementazioni condividono la stessa architettura dati e lo stesso backend Firebase, dimostrando come sia possibile sviluppare applicazioni equivalenti utilizzando tecnologie differenti.

0.6 Conclusioni

L'obiettivo del progetto era lo sviluppo di un'applicazione mobile dedicata alla sicurezza personale, capace di mettere rapidamente a disposizione dell'utente strumenti utili in situazioni di emergenza.

Nel corso del progetto sono state realizzate due implementazioni differenti: una versione nativa Android sviluppata in Kotlin e una versione multiplatforma sviluppata mediante Flutter. Entrambe condividono lo stesso backend Firebase Cloud Firestore, consentendo di utilizzare un'unica struttura dati indipendentemente dalla tecnologia impiegata lato client.

La versione Android ha permesso di approfondire numerosi aspetti dello sviluppo mobile, tra cui la gestione della localizzazione, l'integrazione con Google Maps, l'utilizzo delle Google Directions API, la comunicazione tramite SMS e la realizzazione di funzionalità più articolate come il percorso sicuro, il tracciamento della posizione in tempo reale e il sistema di supporto psicologico.

La versione Flutter ha invece consentito di sperimentare un approccio multiplatforma, evidenziando i vantaggi offerti dal framework nella realizzazione di interfacce moderne e riutilizzabili su sistemi operativi differenti.

Alcune caratteristiche sono state implementate esclusivamente nella versione Android (la versione Flutter è stata sviluppata come dimostrazione delle principali funzionalità dell'applicazione).

Nel complesso il progetto ha consentito di acquisire esperienza nello sviluppo di applicazioni mobile moderne, nell'utilizzo dei servizi cloud di Firebase, nell'integrazione di API esterne e nella progettazione di interfacce orientate all'usabilità in contesti di emergenza.

